

基于 BERT 的文本对抗攻击与鲁棒性分析

杨蕊竹, 姚欣煜, 徐梦茹, 陆鹏宇, 井立帅

方向 10 小组 B

摘要: 随着预训练语言模型在文本分类、内容审核、舆情分析和垃圾信息过滤等软件系统中的广泛部署,模型在对抗场景下的鲁棒性已经成为软件质量保障的重要问题.本文以 textattack/bert-base-uncased-imdb 为目标模型,围绕 IMDB 情感分类任务开展 BERT 文本对抗攻击与鲁棒性分析.首先,基于 TextAttack 框架复现 TextFooler 和 BERT-Attack 两类经典黑盒词级攻击方法,验证 BERT 在少量词替换扰动下的脆弱性.其次,针对 BERT-Attack 候选替换词可能产生语义漂移的问题,实现一种语义约束攻击方法 SC-BERT-Attack,在 BERT-MLM 候选生成后引入 BERT 词嵌入余弦相似度过滤,以提升对抗文本的自然性和语义保真度.最后,通过对抗训练构建鲁棒模型,比较干净模型与鲁棒模型在 TextFooler、BERT-Attack 和 SC-BERT-Attack 下的攻击成功率变化.实验结果表明,TextFooler 和 BERT-Attack 对原始 BERT 模型具有较强攻击能力,攻击成功率分别达到 98.6% 和 91.9%;SC-BERT-Attack 在牺牲部分攻击成功率的同时,将平均查询次数由 944.4 降至 397.1,语义相似度由 0.9905 提升至 0.9923;对抗训练后,三类攻击的 ASR 分别下降至 5.5%、6.1% 和 0.0%,说明对抗训练能够显著提升模型在本实验设置下的鲁棒性.本文形成了从基线复现、方法改进到防御验证的完整实验闭环,为预训练语言模型的软件质量评估提供了可复现的实验参考.

关键词: BERT; 文本对抗攻击; TextFooler; BERT-Attack; SC-BERT-Attack; 对抗训练; 鲁棒性分析

Text Adversarial Attack and Robustness Analysis Based on BERT

YANG Ruizhu, YAO Xinyu, XU Mengru, LU Pengyu, JING Lishuai

Abstract: With the wide deployment of pretrained language models in software systems such as text classification, content moderation and spam filtering, adversarial robustness has become an important concern in software quality assurance. This report investigates word-level adversarial attacks and robustness evaluation on a BERT-based IMDB sentiment classifier. We first reproduce TextFooler and BERT-Attack using TextAttack to verify the vulnerability of BERT under small textual perturbations. Then we implement SC-BERT-Attack, a semantically constrained variant of BERT-Attack that filters BERT-MLM substitution candidates by cosine similarity of BERT word embeddings. Finally, adversarial training is used to evaluate whether a robust model can reduce the attack success rates of three attack methods. Experimental results show that the reproduced TextFooler and BERT-Attack achieve strong ASR on the clean model. SC-BERT-Attack improves query efficiency and semantic fidelity at the cost of lower ASR. After adversarial training, ASR decreases significantly under all three attacks. The project forms an end-to-end experimental pipeline covering attack reproduction, method improvement, evaluation and defense.

Key words: BERT; text adversarial attack; TextFooler; BERT-Attack; SC-BERT-Attack; adversarial training; robustness analysis

1 引言

近年来,以 BERT 为代表的预训练语言模型在自然语言处理任务中取得了显著效果,并逐渐成为实际软件系统中的核心组件.在情感分析、内容审核、智能客服和垃圾信息识别等场景中,模型的输出往往会进一步影响业务决策或安全策略.因此,模型不仅需要在干净测试集上具有较高准确率,还必须在面对异常输入、恶意扰动和对抗样本时保持稳定.

文本对抗攻击研究表明,自然语言模型对细微词级扰动十分敏感.攻击者只需替换输入文本中的少量关键词,就可能使模型输出发生翻转,而人类读者仍然认为文本语义基本不变.这种脆弱性意味着,仅以标准测试准确

率衡量模型质量是不充分的.从软件质量保障角度看,对抗鲁棒性测试可以被视为一种面向机器学习组件的安全测试和可靠性测试:它通过系统构造边界输入来暴露模型在真实部署环境中的潜在失效模式.

本项目开题阶段提出“基于 BERT 的时序感知模型对抗攻击与鲁棒性分析”.其中,“时序感知”并不等同于传统时间序列建模,而是强调文本 token 序列在模型预测过程中的结构性影响.传统 LSTM 通过循环状态建模序列依赖,而 BERT 通过自注意力机制建模全局 token 关系.在攻击场景下,词级替换可以被理解为对关键序列位置的扰动;攻击算法通过词重要性排序或上下文候选生成,寻找最容易导致模型决策边界跨越的 token.

围绕上述目标,本项目最终形成了以下研究主线:

1. 在 IMDB 情感分类任务上复现 TextFooler 和 BERT-Attack,评估原始 BERT 模型的脆弱性.
2. 针对 BERT-Attack 候选词缺乏显式语义过滤的问题,实现 SC-BERT-Attack,提升对抗样本语义保真度和查询效率.
3. 使用对抗训练构建鲁棒模型,比较防御前后三种攻击方法的 ASR 变化,分析鲁棒性提升效果.
4. 汇总实验数据并生成图表,形成可复现的实验平台和报告材料.

需要说明的是,开题报告中曾提到 HotFlip、注意力热力图和动态语义约束等计划内容.从项目实际文件看,最终实现重点转向了黑盒词级攻击复现、固定阈值语义过滤改进和对抗训练防御.因此本文在报告中以实际完成的代码和结果为准,将“动态约束策略”收敛表述为“基于 BERT 词嵌入相似度的语义约束过滤”,避免夸大未落地内容.

2 相关工作

2.1 文本对抗攻击

文本对抗攻击的目标是在尽量保持原始语义和可读性的前提下,构造能够误导目标模型的输入样本.与图像扰动不同,文本是离散符号序列,不能直接在像素空间添加连续噪声.因此,文本攻击通常以字符替换、词替换、短语改写或句法变换为主要形式.

按照攻击者掌握的信息,文本对抗攻击可分为白盒攻击和黑盒攻击.白盒攻击可以访问模型结构、参数和梯度,典型方法如 HotFlip 通过梯度方向估计 token 替换对损失函数的影响.黑盒攻击只能访问模型输入输出接口,更接近真实部署场景.本项目选择 TextFooler 和 BERT-Attack 作为主要基线,正是因为它们属于典型黑盒词级攻击方法,并且可以通过 TextAttack 框架稳定复现.

2.2 TextFooler

TextFooler 是 Jin 等人提出的强黑盒文本攻击方法.其基本流程包括:首先计算每个词对模型预测置信度的影响,得到词重要性排序 Word Importance Ranking;然后为重要词寻找同义替换候选;最后在词性一致、语义相似度和预测翻转等约束下逐步替换词语.TextFooler 的优势在于实现简单、攻击成功率高,能够作为评估文本分类模型鲁棒性的强基线.

在本项目中,TextFooler 由 attack/baseline_attack.py 调用 TextAttack 的 TextFoolerJin2019 recipe 实现.实验结果输出至 results/baseline/textfooler_results.csv,并由 evaluate/evaluate.py 汇总至最终结果表.

2.3 BERT-Attack

BERT-Attack 是 Li 等人提出的面向 BERT 的黑盒攻击方法.与 TextFooler 依赖静态词向量或同义词词典不同,BERT-Attack 利用 BERT 的 Masked Language Model 生成上下文相关候选词.例如,在一个句子中将某个词替换为 [MASK],再由 BERT-MLM 根据上下文预测候选 token.这种候选生成方式通常比静态同义词表更自然,也更能适应上下文语境.

项目中 BERT-Attack 同样由 attack/baseline_attack.py 调用 TextAttack 的 BERTAttackLi2020 recipe 实现.由于 BERT-Attack 计算成本较高,代码中加入了 max_candidates、max_percent、单样本超时和 checkpoint

恢复机制,以保证长时间实验可以分批完成.

2.4 对抗样本质量评估

早期文本攻击研究往往强调攻击成功率,但后续研究指出,如果对抗文本语义已经严重改变,即使模型预测翻转也不能说明攻击样本质量高.因此,对抗攻击评估应同时考虑 ASR、修改比例、查询次数、语义相似度和人工可读性等指标.

本项目采用如下指标:

- 1. **ASR(Attack Success Rate):** 攻击成功样本数占有效攻击样本数的比例.对于 TextAttack 输出,本文采用 Successful / (Successful + Failed),排除模型原本预测错误的 Skipped 样本.
- 2. **Avg Queries:** 平均模型查询次数,反映攻击效率和攻击成本.
- 3. **Avg Words Changed 或 Perturb Rate:** 文本修改程度.对于 TextAttack CSV,若未输出修改词数则记为空;对于自实现攻击,使用修改词数占总词数比例.
- 4. **Semantic Similarity:** 原文和对抗文本的语义相似度.项目中主要使用 sentence-transformers 的 all-MiniLM-L6-v2 计算成功样本的句向量余弦相似度.

2.5 对抗训练防御

对抗训练是提升模型鲁棒性的经典方法.其核心思想是在训练过程中加入对抗样本,使模型学习到更稳定的决策边界.对于文本分类任务,可以先使用攻击算法生成能够误导模型的文本,再将这些样本与原始样本混合训练.对抗训练通常会牺牲部分干净准确率,但可以显著降低模型在相似攻击下的 ASR.

本项目中的防御代码位于 defense/adversarial_training.py.该脚本使用 TextFooler 从训练集中生成对抗样本,再将干净样本和对抗样本按近似 1:1 比例混合,对 BERT 进行 2 epoch 微调,最后用三类攻击方法评估鲁棒模型.

3 项目实现与实验平台

3.1 项目目录结构

项目主体位于 Desktop/ruanzhi 目录.核心结构如下:

路径	功能
configs/config.py	全局配置,包括模型名、数据集、路径和默认攻击样本数
data/download_data.py	预下载数据集和 NLTK 资源
train/finetune_bert.py	BERT 微调参考脚本,实际攻击主要使用 TextAttack 官方模型
attack/baseline_attack.py	TextFooler 与 BERT-Attack 复现实验
attack/improved_attack.py	SC-BERT-Attack 改进攻击
defense/adversarial_training.py	对抗训练防御
evaluate/evaluate.py	汇总 baseline、改进攻击和防御结果
evaluate/visualize.py	生成 ASR、查询次数、防御效果等图表
results/	存放 CSV、JSON 和可视化图表

3.2 实验模型与数据集

本项目使用的受害模型为:

textattack/bert-base-uncased-imdb

该模型是 TextAttack 官方发布的在 IMDB 情感分类数据集上微调后的 BERT-base-uncased 模型,干净准确率约为 93.7%.使用该模型可以跳过本地训练,降低环境依赖和硬件门槛,更适合作为课程项目的统一受害模型.

数据集使用 Hugging Face Datasets 中的 imdb 数据集,任务为二分类情感分析.实验中 TextFooler 使用 IMDB 测试集前 1000 条样本,BERT-Attack 使用测试集前 200 条样本,SC-BERT-Attack 使用测试集前 200

条样本.不同方法样本数不完全一致,主要由算法运行成本决定.

3.3 实验环境与依赖

项目主要依赖如下:

类别	工具或库
深度学习框架	PyTorch
模型库	Hugging Face Transformers
数据集库	Hugging Face Datasets
攻击框架	TextAttack
指标计算	sentence-transformers, scikit-learn
可视化	matplotlib, seaborn
语言资源	NLTK

项目提供了 `install.py` 和 `main.py` 用于环境安装与检查.其中 `main.py` 会验证 PyTorch、Transformers 和 TextAttack 是否可用.在实际运行中,由于 Universal Sentence Encoder 依赖 TensorFlow/TFHub,部分脚本对 USE 约束或 TensorFlow GPU 占用进行了规避,以减少 Windows 或离线环境下的卡死和显存冲突问题.

3.4 结果汇总流程

项目结果文件主要包括:

文件	内容
results/baseline/textfooler_results.csv	TextFooler 原始攻击结果
results/baseline/bertattack_results.csv	BERT-Attack 原始攻击结果
results/improved/improved_results.json	SC-BERT-Attack 与自实现 BERT-Attack 对照结果
results/defense/baseline/*.csv	鲁棒模型上的 TextFooler 和 BERT-Attack 攻击结果
results/defense/improved/improved_results.json	鲁棒模型上的 SC-BERT-Attack 结果
results/final_comparison.csv	综合攻击对比表
results/defense_comparison.csv	干净模型与鲁棒模型防御对比表
results/figures/*.png	可视化图表

其中 `evaluate/evaluate.py` 负责统一读取 CSV 和 JSON,计算 ASR、平均查询次数、语义相似度等指标,并输出最终对比表.`evaluate/visualize.py` 则根据汇总表生成图表.

4 基线攻击复现实验

4.1 TextFooler 复现设置

TextFooler 实验使用 `attack/baseline_attack.py --attack textfooler` 运行.受害模型为 `textattack/bert-base-uncased-imdb`,数据为 IMDB test split 前 1000 条样本.攻击结果保存至 `results/baseline/textfooler_results.csv`.

TextFooler 的攻击流程可以概括为:

1. 对原始样本进行模型预测,跳过模型原本分类错误的样本.
2. 删除每个词并观察目标类别置信度下降幅度,得到词重要性排序.
3. 对重要词生成候选替换词.
4. 按语义和语法约束筛选候选词.
5. 逐步替换,直至目标模型分类错误或候选空间耗尽.

4.2 BERT-Attack 复现设置

BERT-Attack 实验使用 `attack/baseline_attack.py --attack bertattack` 运行.考虑到 BERT-Attack 需要频繁调用 MLM 候选生成和目标模型查询,本项目使用 200 条测试样本进行复现.参数包括 `max_candidates=8`, `max_percent=0.4`,单样本超时限制 300 秒.实验结果保存至 `results/baseline/bertattack_results.csv`.

项目额外实现了分批运行脚本 `attack/run_bertattack_batches.py`,并在 `baseline_attack.py` 中加入 `checkpoint` 恢复逻辑.这类工程处理虽然不改变算法本身,但对长时间攻击实验的可复现性很重要.

4.3 复现结果

表 1 给出两种经典攻击方法的复现实验结果.

表 1 基线攻击复现结果							
方法	样本总数	成功数	失败数	跳过数	ASR(%)	平均查询次数	语义相似度
TextFooler	1000	930	13	57	98.6	652.5	0.9266
BERT-Attack	200	171	15	14	91.9	267.2	0.9295

需要特别说明的是,表中 ASR 按 $\text{Successful} / (\text{Successful} + \text{Failed})$ 计算,即排除 Skipped 样本.对于 TextFooler,若按 $\text{Successful} / \text{Total}$ 计算,则 ASR 为 93.0%.为避免口径不一致,本文后续统一采用排除跳过样本的 ASR.

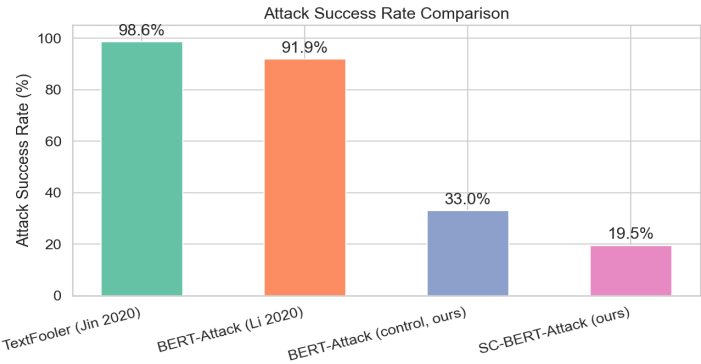


图 1 原始模型下不同攻击方法的攻击成功率对比

从结果看,TextFooler 和 BERT-Attack 均能有效攻击 BERT-IMDB 模型.TextFooler 在有效样本上的 ASR 达到 98.6%,说明词级替换对 BERT 情感分类模型具有很强破坏力.BERT-Attack 的 ASR 为 91.9%,与论文中 BERT-IMDB 任务上超过 90% 的攻击成功率相符.同时,BERT-Attack 的平均查询次数为 267.2,明显低于 TextFooler 的 652.5,说明上下文感知候选词在本实验设置下具有较高搜索效率.

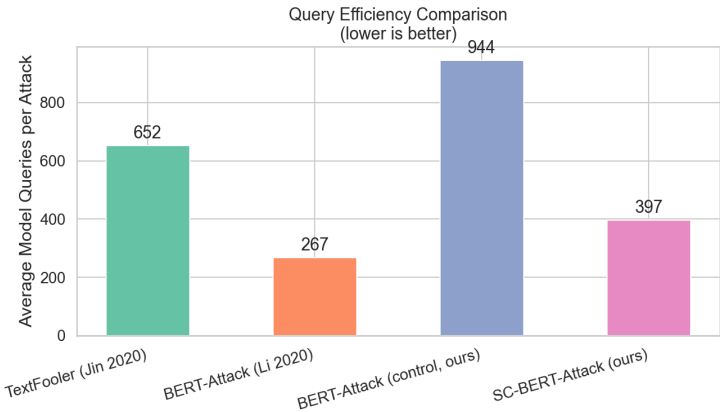


图 2 不同攻击方法的平均查询次数对比

5 SC-BERT-Attack 改进方法

5.1 改进动机

标准 BERT-Attack 使用 BERT-MLM 生成候选词,可以获得上下文相关替换.但是,MLM 的目标是预测上

下文中可能出现的词,并不直接保证替换词与原词语义一致.在情感分类任务中,某些候选词虽然符合局部语法,却可能改变句子情感倾向.例如,原句中的 `positive sentiment word` 若被替换为相反情感词,模型预测翻转并不能说明攻击样本保持了原始语义.

因此,本项目在 BERT-Attack 的候选生成阶段加入词级语义过滤,提出 SC-BERT-Attack.其中 SC 表示 Semantically-Constrained,即语义约束.该方法不追求单纯提高 ASR,而是尝试在攻击成功率、语义保真和查询效率之间取得更平衡的结果.

5.2 方法设计

SC-BERT-Attack 的核心流程如下:

1. 对原始文本进行目标模型预测,若模型原本预测错误则跳过.
2. 使用删除评分计算每个词的重要性,得到 WIR 排序.
3. 对当前重要词使用 BERT-MLM 生成 top-k 候选替换词.
4. 计算候选词与原词在 BERT embedding 空间中的余弦相似度.
5. 保留相似度不低于 0.5 的候选词;若过滤后为空,回退到前 3 个原始候选词以避免候选空间完全消失.
6. 批量查询目标模型,若某个替换使预测类别翻转,则攻击成功.

候选过滤可表示为:

$$C_{\text{sem}} = \{w \in \text{MLM_topK}(x_i) \mid \cos(\text{emb}(w), \text{emb}(x_i)) \geq 0.5\} \quad (1)$$

其中 $\text{emb}(w)$ 表示 BERT 词嵌入层中 token w 的向量表示.项目实现中,MLM 模型使用 `bert-base-uncased`,受害分类器使用 `textattack/bert-base-uncased-imdb`.为了节省显存,改进攻击脚本将 MLM 模型保留在 CPU 上.

5.3 实验结果

改进攻击实验在 200 条 IMDB 测试样本上运行,对比自实现 BERT-Attack control 与 SC-BERT-Attack.结果见表 2.

表 2 SC-BERT-Attack 与对照组比较

指标	BERT-Attack(control)	SC-BERT-Attack	变化
成功样本数	66 / 200	39 / 200	-27
ASR(%)	33.0	19.5	-13.5 pp
平均查询次数	944.4	397.1	-58.0%
扰动率	0.95%	0.85%	-0.10 pp
语义相似度	0.9905	0.9923	+0.0018

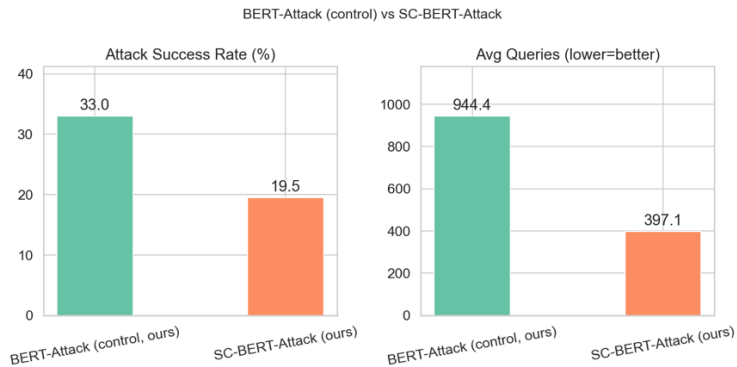


图 3 自实现 BERT-Attack 与 SC-BERT-Attack 对比

结果显示,SC-BERT-Attack 的 ASR 从 33.0% 下降至 19.5%,这说明语义过滤确实缩小了搜索空间,减少了可用于攻击的候选替换词.然而,平均查询次数从 944.4 降至 397.1,下降约 58.0%,说明过滤后的候选池更小、更集中,攻击过程不再需要测试大量低质量候选.同时,扰动率从 0.95% 降至 0.85%,语义相似度从 0.9905 提升至 0.9923,说明该方法生成的成功对抗样本更接近原文.

因此,SC-BERT-Attack 的价值不在于获得最高 ASR,而在于提供一种面向高质量对抗样本的攻击策略.在需要强调隐蔽性、语义一致性和查询成本的场景中,该方法比单纯追求攻击成功率的基线更有分析意义.

6 对抗训练防御实验

6.1 防御方法

对抗训练脚本位于 defense/adversarial_training.py.其基本思路为:

- 1. 加载原始 BERT-IMDB 模型.
- 2. 使用 TextFooler 攻击训练集样本,生成成功对抗样本.
- 3. 将对抗样本与相同规模的干净样本混合.
- 4. 在混合数据集上继续微调 BERT,训练 2 epoch.
- 5. 保存鲁棒模型,再使用 TextFooler、BERT-Attack 和 SC-BERT-Attack 评估防御效果.

对抗训练可以被理解作为一种数据增强式防御方法.模型在训练过程中接触到被攻击后的文本,从而学习到更平滑的决策边界.不过,这类方法通常会带来一定代价:一方面训练成本增加;另一方面,模型在干净数据上的准确率可能略有下降.本项目仓库中包含防御评估结果,但未提交完整鲁棒模型权重目录,因此本文重点分析结果文件中已保存的防御效果.

6.2 防御结果

表 3 展示干净模型与鲁棒模型在三种攻击方法下的 ASR 对比.

表 3 对抗训练前后攻击成功率对比			
攻击方法	干净模型 ASR(%)	鲁棒模型 ASR(%)	下降幅度
TextFooler	98.6	5.5	-93.1 pp
BERT-Attack	91.9	6.1	-85.8 pp
SC-BERT-Attack	19.5	0.0	-19.5 pp

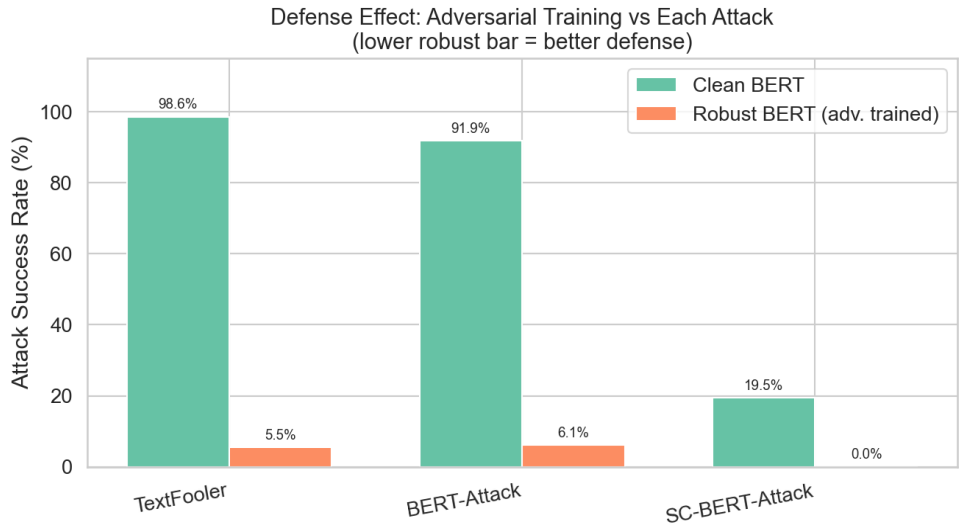


图 4 对抗训练防御效果对比

从表 3 可见,对抗训练显著降低了三类攻击在鲁棒模型上的成功率.TextFooler 的 ASR 从 98.6% 降至 5.5%,BERT-Attack 从 91.9% 降至 6.1%,SC-BERT-Attack 则未能在 200 条样本上成功攻击鲁棒模型.同时,防御结果表明鲁棒模型上的平均查询次数明显上升:TextFooler 从 652.5 增至 2887.7,BERT-Attack 从 267.2 增至 790.6,SC-BERT-Attack 在鲁棒模型上平均查询次数为 506.1.这说明即使攻击没有成功,攻击算法也需要消耗更多查询才能搜索候选空间,体现出一定“成本阻吓”效果.

表 4 给出防御结果的完整指标.

表 4 干净模型与鲁棒模型完整防御指标

方法	样本数	成功数	失败数	跳过数	ASR (%)	平均查询次数	语义相似度
Clean BERT + TextFooler	1000	930	13	57	98.6	652.5	0.9266
Robust BERT + TextFooler	200	11	189	0	5.5	2887.7	0.7897
Clean BERT + BERT-Attack	200	171	15	14	91.9	267.2	0.9295
Robust BERT + BERT-Attack	164	10	154	0	6.1	790.6	0.7696
Clean BERT + SC-BERT-Attack	200	39	161	0	19.5	397.1	0.9923
Robust BERT + SC-BERT-Attack	200	0	200	0	0.0	506.1	-

需要注意,鲁棒模型上的成功样本语义相似度较低,这可能意味着攻击算法在少数成功样本上不得不采用更激进的替换才能绕过防御.对于 SC-BERT-Attack,由于无成功样本,语义相似度无法计算.

7 综合结果与讨论

7.1 综合对比

表 5 汇总原始模型上的四组攻击结果.

表 5 原始模型综合攻击结果

方法	样本总数	成功数	失败数	跳过数	ASR (%)	扰动/修改指标	平均查询次数	语义相似度
TextFooler (Jin 2020)	1000	930	13	57	98.6	-	652.5	0.9266
BERT-Attack (Li 2020)	200	171	15	14	91.9	-	267.2	0.9295
BERT-Attack (control, ours)	200	66	134	0	33.0	0.010	944.4	0.9905
SC-BERT-Attack (ours)	200	39	161	0	19.5	0.009	397.1	0.9923

从综合结果看,TextAttack 框架复现的 TextFooler 和 BERT-Attack 代表了强攻击基线,适合用于验证模型脆弱性.自实现的 BERT-Attack control 与 TextAttack 官方 recipe 不完全一致,其 ASR 明显更低,原因可能包括候选数量、搜索策略、约束设置、文本截断长度和实现细节差异.因此,报告中不应将自实现 control 与官方 BERT-Attack 直接视为同一实现,而应把它作为 SC-BERT-Attack 的 A/B 对照组.

7.2 攻击成功率与语义保真的权衡

TextFooler 与 TextAttack 版 BERT-Attack 在 ASR 上表现最强,但它们的成功样本语义相似度约为 0.93.SC-BERT-Attack 的 ASR 较低,但语义相似度达到 0.9923.这反映出文本攻击中的典型权衡:若允许更大搜索空间和更自由替换,模型更容易被攻击,但对抗文本可能出现语义漂移;若加强语义过滤,样本质量更好,但攻击成功率下降.

对于软件质量保障而言,这两类指标都有意义.高 ASR 攻击用于暴露模型最坏情况下的脆弱性;高语义保真攻击用于评估模型在真实隐蔽攻击下的风险.因此,本项目同时保留基线攻击和改进攻击是合理的.

7.3 查询效率分析

查询次数反映黑盒攻击成本.在真实部署环境中,攻击者通常只能通过接口访问模型,查询次数越多,越容易被监控系统发现或触发速率限制.从结果看,BERT-Attack 的平均查询次数低于 TextFooler,说明上下文候选生成对搜索效率有帮助.SC-BERT-Attack 相对于自实现 control 查询次数下降 58.0%,说明语义过滤不仅提升样本质量,也能减少无效候选测试.

对抗训练后,攻击查询次数大幅增加,尤其 TextFooler 达到 2887.7.这说明鲁棒模型不仅降低攻击成功概

率,还提高了攻击成本,这对实际防御具有价值.

8 结论与展望

本文围绕 BERT 文本分类模型的对抗攻击与鲁棒性分析开展实验研究.首先,在 IMDB 情感分类任务上复现 TextFooler 与 BERT-Attack,证明 BERT 虽然在干净数据上具有较高准确率,但面对词级对抗扰动时仍然十分脆弱.TextFooler 和 BERT-Attack 在原始模型上的 ASR 分别达到 98.6% 和 91.9%,说明对抗鲁棒性是预训练语言模型质量保障中不可忽视的问题.

其次,本文实现 SC-BERT-Attack,在 BERT-MLM 候选生成后增加 BERT 词嵌入语义过滤.实验显示,该方法虽然使 ASR 从 33.0% 降至 19.5%,但平均查询次数下降 58.0%,扰动率略有降低,语义相似度略有提升.这表明语义约束过滤能够在攻击效率和样本质量方面带来收益,适合用于高质量对抗样本生成分析.

最后,本文通过对抗训练验证防御效果.结果表明,对抗训练后 TextFooler、BERT-Attack 和 SC-BERT-Attack 的 ASR 分别降至 5.5%、6.1% 和 0.0%,攻击查询成本显著增加.这说明对抗训练在本项目实验设置下能够有效提升模型鲁棒性.

后续工作可以从以下方向展开:

1. 实现真正的动态语义约束策略,系统比较不同阈值调度方式对 ASR、查询次数和语义相似度的影响.
2. 增加 HotFlip、PWWS、DeepWordBug 等攻击方法,构建更完整的鲁棒性测试基准.
3. 引入人工评估或大语言模型评估,更准确衡量对抗文本可读性和语义保持程度.
4. 在更多数据集和任务上验证方法,例如 AG News、SST-2、自然语言推理和问答任务.
5. 保存并发布鲁棒模型权重和训练日志,提升实验可复现性.
6. 结合注意力可视化和错误案例分析,进一步解释 BERT 在关键 token 扰动下的失效机制.

References

- [1] Jin D, Jin Z, Zhou JT, Szolovits P. Is BERT really robust? A strong baseline for natural language attack on text classification and entailment. In: Proc. of the 34th AAAI Conf. on Artificial Intelligence. Palo Alto: AAAI Press, 2020. 8018-8025.
- [2] Li L, Ma R, Guo Q, Xue X, Qiu X. BERT-Attack: Adversarial attack against BERT using BERT. In: Proc. of the 2020 Conf. on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, 2020. 6193-6202.
- [3] Morris JX, Lifland E, Yoo JY, Grigsby J, Jin D, Qi Y. TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP. In: Proc. of the 2020 Conf. on Empirical Methods in Natural Language Processing: System Demonstrations. Association for Computational Linguistics, 2020. 119-126.
- [4] Ebrahimi J, Rao A, Lowd D, Dou D. HotFlip: White-box adversarial examples for text classification. In: Proc. of the 56th Annual Meeting of the Association for Computational Linguistics. Association for Computational Linguistics, 2018. 31-36.
- [5] Morris JX, Lifland E, Yoo JY, Qi Y. Searching for a search method: Benchmarking search algorithms for generating NLP adversarial examples. In: Proc. of the 2020 Conf. on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, 2020. 1766-1779.
- [6] Morris JX, Lifland E, Yoo JY, Qi Y. Reevaluating adversarial examples in natural language. In: Findings of the Association for Computational Linguistics: EMNLP 2020. Association for Computational Linguistics, 2020. 3829-3839.
- [7] Devlin J, Chang MW, Lee K, Toutanova K. BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proc. of NAACL-HLT. Association for Computational Linguistics, 2019. 4171-4186.